Project 2 Report

3R Arm Simulation Under Gravity

Amal S Pradeep
December 11, 2025
Video link

I. Introduction

This project aims to simulate the dynamics of an open chain 3R robot arm with mass located at the tip of each link and acted upon by gravity. This is an extension of the forward kinematics of a 2R arm and follows the formula: $M(\theta)\theta'' = \tau(t) - h(\theta, \theta') - JT(\theta) F_{tip}$. For the purposes of this project we assume that there is no joint friction and assume only gravity acts on the arm. Hence the torque at each joint is also assumed to be negligible. Under these conditions, the equation of kinematics simplifies to $M(\theta)\theta'' = -C(\theta,\theta)\dot{\theta} - G(\theta)$.

II. Configuration

The robot is initially placed horizontally with all joint angles set to 0°. Joint 1 is the base of the system with joint 2 and joint 3 towards the right-hand side of the coordinate system. Each mass is located at the tip of the links. The mass and length of the arms are set to unity. The robot starts from rest influenced only by gravity. The code involves the following core components: i) A function to compute Mass Matrix, ii) A function to compute gravity and Coriolis terms, ii) A Forward Dynamics function that solves for $\ddot{\theta}$, iii) A numerical integrator function (Euler), iv) An animation.

III. Theory

Forward dynamics equation: $M(\theta)\dot{\theta} = -h(\theta, \dot{\theta})$ where $h(\theta, \dot{\theta}) = c(\theta, \dot{\theta}) + g(\theta)$ contains the Coriolis / centripetal and gravity terms. Position of each joint of the 3R arm is given by-

$$x1 = L1cos\theta1$$
$$y1 = L1sin\theta1$$

$$x2 = L1cos\theta1 + L2cos(\theta1 + \theta2)$$
$$y2 = L1sin\theta1 + L2sin(\theta1 + \theta2)$$

$$x3 = L1cos\theta1 + L2cos(\theta1 + \theta2) + L3cos(\theta1 + \theta2 + \theta3)$$
$$y3 = L1sin\theta1 + L2sin(\theta1 + \theta2) + L3sin(\theta1 + \theta2 + \theta3)$$

Velocity for each joint is given by-

$$\dot{x}1 = -L1sin(\theta1)\dot{\theta}$$
$$\dot{y}1 = L1cos(\theta1)\dot{\theta}1$$

$$\dot{x}2 = -L1sin(\theta1)\dot{\theta}1 - L2sin(\theta1 + \theta2)(\dot{\theta}1 + \dot{\theta}2)$$
$$\dot{y}2 = L1cos(\theta1)\dot{\theta}1 + L2cos(\theta1 + \theta2)(\dot{\theta}1 + \dot{\theta}2)$$

$$x\dot{}3 = -L1sin(\theta1)\theta\dot{}1 - L2sin(\theta1+\theta2)(\theta\dot{}1+\theta\dot{}2) - L3sin(\theta1+\theta2+\theta3)(\theta'1+\theta'2+\theta'3)$$
$$y\dot{}3 = L1cos(\theta1)\theta\dot{}1 + L2cos(\theta1+\theta2)(\theta\dot{}1+\theta\dot{}2) + L3cos(\theta1+\theta2+\theta3)(\theta'1+\theta'2+\theta'3)$$

Kinetic Energy:

$$K_i = \frac{1}{2}m_i(\dot{x}_i^2 + \dot{y}_i^2)$$

The total kinetic energy is $K = K_1 + K_2 + K_3$

For mass 1: $\dot{x}_1^2 + \dot{y}_1^2 = L_1^2\sin^2(\theta_1)\dot{\theta}_1^2 + L_1^2\cos^2(\theta_1)\dot{\theta}_1^2 = L_1^2\dot{\theta}_1^2$

For mass 2: $\dot{x}_2{}^2 + \dot{y}_2{}^2 = [L_1sin(\theta_1)\theta_1 + L_2sin(\theta_1+\theta_2)(\theta_1+\theta_2)]^2 + [L_1cos(\theta_1)\theta_1 + L_2cos(\theta_1+\theta_2)(\theta_1+\theta_2)]^2$

Upon simplification we get-

$$K_2 = \tfrac{1}{2}m_2\{[L_1{}^2 + L_2{}^2 + 2L_1L_2cos(\theta_2)]\theta_1{}^2 + L_2{}^2\theta_2{}^2 + [2L_2{}^2 + 2L_1L_2cos(\theta_2)]\theta_1\theta_2\}$$

For mass 3: $\dot{x}_3{}^2 + \dot{y}_3{}^2 = [L_1sin(\theta_1)\theta_1 + L_2sin(\theta_1+\theta_2)(\theta_1+\theta_2) + L_3sin(\theta_1+\theta_2+\theta_3)(\theta_1+\theta_2+\theta_3)]^2 + [L_1cos(\theta_1)\theta_1 + L_2cos(\theta_1+\theta_2)(\theta_1+\theta_2) + L_3cos(\theta_1+\theta_2+\theta_3)(\theta_1+\theta_2+\theta_3)]^2$

Upon simplification we get-

$$K_3 = \tfrac{1}{2}m_3\{[L^{12} + L^{22} + L^{32} + 2L^1L^2\cos(\theta^2) + 2L^1L^3\cos(\theta^2+\theta^3) + 2L^2L^3\cos(\theta^3)]\dot{\theta}^{12} + [L_2{}^2 + L_3{}^2 + 2L_2L_3\cos(\theta_3)]\theta\dot{}_2{}^2 + L_3{}^2\theta\dot{}_3{}^2 + [2L_2{}^2 + 2L_3{}^2 + 2L_1L_2\cos(\theta_2) + 2L_1L_3cos(\theta_2+\theta_3) + 4L_2L_3cos(\theta_3)]\theta_1\theta_2 + [2L_3{}^2 + 2L_1L_3cos(\theta_2+\theta_3) + 2L_2L_3cos(\theta_3)]\theta_1\theta_3 + [2L_3{}^2 + 2L_2L_3cos(\theta_3)]\theta_2\theta_3\}$$

Potential Energy:

For gravity acting in the $-\hat{y}$direction: $P = m_1gy_1 + m_2gy_2 + m_3gy_3$

With parameters ($m_i = L_i = 1$ and keeping g as it is):

$$P = g[\sin\theta_1 + \sin\theta_1 + \sin(\theta_1+\theta_2) + \sin\theta_1 + \sin(\theta_1+\theta_2) + \sin(\theta_1+\theta_2+\theta_3)]$$

$$P = g[3\sin\theta_1 + 2\sin(\theta_1+\theta_2) + \sin(\theta_1+\theta_2+\theta_3)]$$

Joint 1 is the "base" of the entire chain which when rotated, moves all three masses. Joint 2 moves masses 2 and 3, and joint 3 only moves mass 3. This hierarchical structure shows up throughout the dynamics. Hence the $3\sin\theta_1$ term. This shows there is more dependence on the first joint angle than the subsequent joint angles.

For the code we use gravity vector which is: $g_i = \frac{\partial P}{\partial \theta_i}$

Therefore,

$$g_1 = \frac{\partial P}{\partial \theta_1} = g[3\cos\theta_1 + 2\cos(\theta_1+\theta_2) + \cos(\theta_1+\theta_2+\theta_3)]$$

$$g_2 = \frac{\partial P}{\partial \theta_2} = g[2\cos(\theta_1 + \theta_2) + \cos(\theta_1 + \theta_2 + \theta_3)]$$
$$g_3 = \frac{\partial P}{\partial \theta_3} = g[\cos(\theta_1 + \theta_2 + \theta_3)]$$

The Equations of Motion:

We use Euler-Lagrange equation:

$$\tau_i = \frac{d}{dt}\frac{\partial L}{\partial \dot{\theta}_i} - \frac{\partial L}{\partial \theta_i}$$

where $L = K - P$ is the Lagrangian.

This yields the familiar form: $\tau = M(\theta)\ddot{\theta} + c(\theta, \dot{\theta}) + g(\theta)$. For this particular simulation with **no joint torques** (τ = 0), the forward dynamics becomes: $\ddot{\theta} = M^{-1}(\theta)\left[-c(\theta, \dot{\theta}) - g(\theta)\right]$

The Mass Matrix M(θ)

For a 3R arm, M is 3×3 and symmetric as it should follow: $KE = \frac{1}{2}\dot{\theta}^T M(\theta)\dot{\theta}$. To obtain the mass matrix we follow a set of assumptions like each mass being a point mass and being located at the end of a link. With this we write the forward kinematics with positions of each joint. Then we compute the Jacobians for each point mass where we obtain a 2x3 matrix. From there we follow up with this formula to find each column of the mass matrix: $M(q) = \sum m_i J_i^T J_i$.

The Gravity Vector g(θ)

This comes from $\frac{\partial P}{\partial \theta_i}$. Taking derivatives of potential energy. From this, the gravity vector is:

$$\mathbf{g}(\boldsymbol{\theta}) = \left[\frac{\partial U}{\partial \theta_1} \quad \frac{\partial U}{\partial \theta_2} \quad \frac{\partial U}{\partial \theta_3}\right]^{\mathrm{T}}$$

The final Gravity vector for 3R arm for unit mass is given by-

$$\begin{aligned} g_1 &= g[L_1\cos\, q_1 + L_1\cos\, q_1 + L_2\cos(q_1 + q_2) + L_1\cos\, q_1 + L_2\cos(q_1 + q_2) + L_3\cos(q_1 + q_2 + q_3)] \\ g_2 &= g[L_2\cos(q_1 + q_2) + L_2\cos(q_1 + q_2) + L_3\cos(q_1 + q_2 + q_3)] \\ g_3 &= g[L_3\cos(q_1 + q_2 + q_3)] \end{aligned}$$

Which simplifies to-

$$\begin{aligned} g_1 &= g(3\cos\theta_1 + 2\cos(\theta_1 + \theta_2) + \cos(\theta_1 + \theta_2 + \theta_3)) \\ g_2 &= g(2\cos(\theta_1 + \theta_2) + \cos(\theta_1 + \theta_2 + \theta_3)) \\ g_3 &= \mathrm{g}\cos(\theta_1 + \theta_2 + \theta_3) \end{aligned}$$

The Coriolis/Centripetal Terms c ($\theta, \dot{\theta}$)

These are the velocity-dependent terms and they arise from the $\frac{d}{dt}\frac{\partial K}{\partial \dot{\theta}_i}$part of the Euler-Lagrange equations. They're quadratic in $\dot{\theta}$ and involve products like $\dot{\theta}_1\dot{\theta}_2$, $\dot{\theta}_2^2$, etc. It is computed from working through the partial derivatives of M(θ): $\boldsymbol{C(q,\dot{q})\dot{q}} = \frac{1}{2}\sum_{i,j,k} \boldsymbol{\Gamma_{ijk}\,(q)\dot{q}_j\dot{q}_k}$,where the Christoffel symbols come from: $\Gamma_{ijk} = \frac{1}{2}\left(\frac{\partial M_{ik}}{\partial q_j} + \frac{\partial M_{ij}}{\partial q_k} - \frac{\partial M_{jk}}{\partial q_i}\right)$. A nested loop is used to compute this value for simulation.

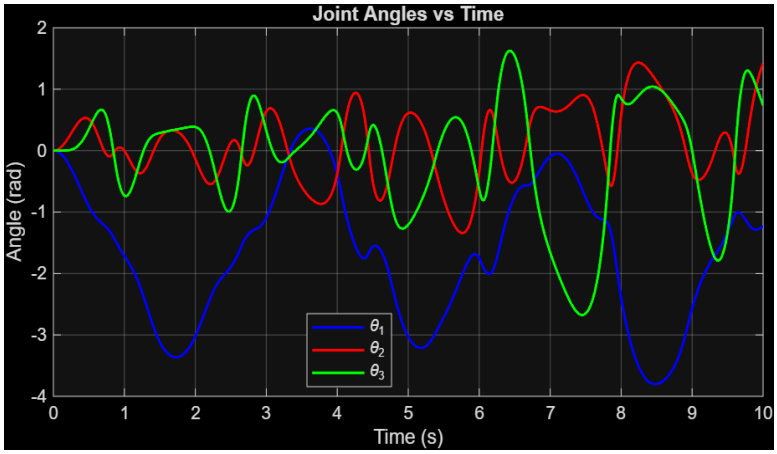
IV. RESULT AND INFERENCE

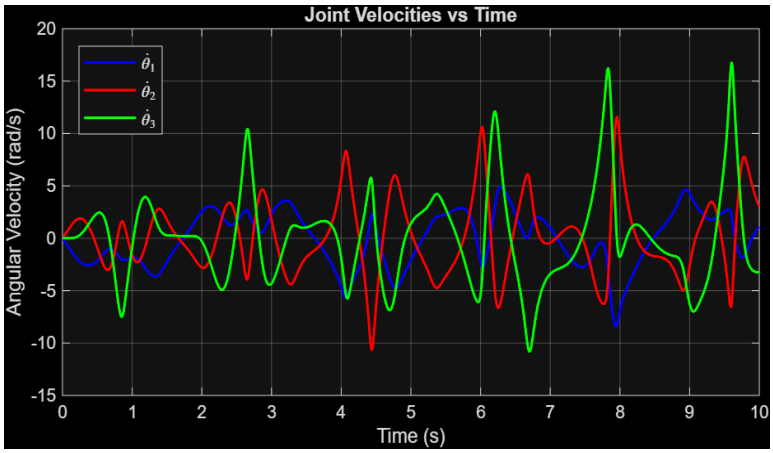
The simulation has resulted in expected results. The joint angles plot indicate Θ1 having the highest deviation at first while the other two joint angles trail back. This concurs with simulation. The overall variation of Θ1 indicate a periodic pendulum like motion.

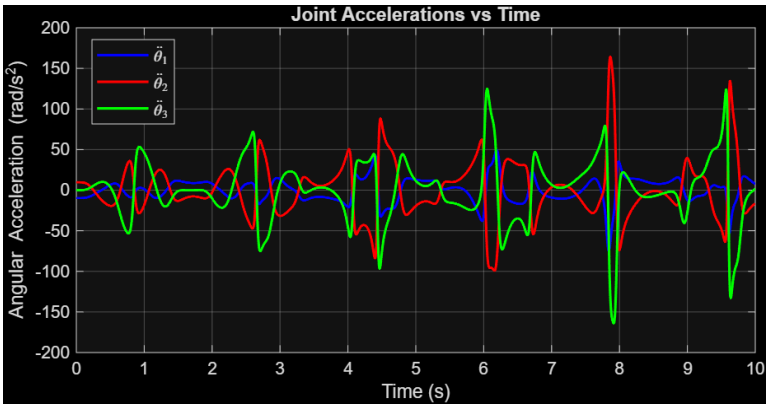
For joint velocities, all joint velocities start at zero and increase as gravity is being acted. They follow the direction of angular acceleration which shows that only joint one has negative velocity. This is because inertia keeps the other two masses fairly level relative to each other. Hence only joint one initially moves downward while other two relative to joint 1 moves upwards.

The joint acceleration is also as expected where joint 1 has an acceleration value ~-9.8 m/s^2, which concurs with what's expected whereas joint 2 has a positive acceleration of ~9.8 m/s^2. Intuitively speaking, this is due to the inertia seen in the point of view of joint 1. Therefore it concurs with what's expected. And as such joint 3 should be experiencing no acceleration relative to joint 2 due to the combined effect of gravity and acceleration of joint2 in the positive direction.

The simulation is showcased in the video linked next to the title of the report.







V. Appendix

```
%Simulation of 3R arm falling under gravity under no friction or torque

function simulate_3R_falling()
    %Setting up parameters
    g = 9.81;
    dt = 0.002;         %timestep
    t_final = 10;        %total simulation time
    t = 0:dt:t_final;
    N = length(t);

    %Setting Initial conditions
    q = [0; 0; 0]; %theta=q
    q_d = [0; 0; 0]; %angular velocity

    %Allocating storage for theta: Is useful in computation
    q_hist = zeros(3, N);
    q_d_hist = zeros(3, N);
    q_dd_hist = zeros(3, N);


    %Simulation using Euler integration
    for i = 1:N
        %Store current state
        q_hist(:,i) = q;
        q_d_hist(:,i) = q_d;

        %Defining
        M = mass_matrix(q);  %Mass Matrix
        grav = gravity_vector(q, g);  %gravity term
        cor = coriolis_vector(q, q_d);  %Coriolis term

        %Forward dynamics: [ M * q_dd = -c - g], for torque=0
        q_dd = M\(-cor - grav); %Solves for theta in terms of above equation
        q_dd_hist(:,i) = q_dd; %Saves value of theta obtained to history

        %Euler integration
        q = q + q_d * dt; %gives us the next value for position
        q_d = q_d + q_dd * dt;  %gives us next value for angular velocity
    end

    % Plot results
    plot_results(t, q_hist, q_d_hist, q_dd_hist);

    % Animation
    animate_arm(t, q_hist, dt);
end

%Mass matrix
function M = mass_matrix(q)
    q1 = q(1); q2 = q(2); q3 = q(3);

    % Link lengths and masses where all equals 1
    L1=1; L2=1; L3=1;
    m1=1; m2=1; m3=1;

    %Jacobian for mass at end of link 1
    J1 = [ -L1*sin(q1), 0, 0;
            L1*cos(q1), 0, 0];

    %Jacobian for mass at end of link 2
    J2 = [ -L1*sin(q1)-L2*sin(q1+q2), -L2*sin(q1+q2), 0;
            L1*cos(q1)+L2*cos(q1+q2), L2*cos(q1+q2), 0];
```

```
    %Jacobian for mass at end of link 3
    J3 = [ -L1*sin(q1)-L2*sin(q1+q2)-L3*sin(q1+q2+q3), -L2*sin(q1+q2)-L3*sin(q1+q2+q3), -
L3*sin(q1+q2+q3);
           L1*cos(q1)+L2*cos(q1+q2)+L3*cos(q1+q2+q3), L2*cos(q1+q2)+L3*cos(q1+q2+q3),
L3*cos(q1+q2+q3) ];

    %Mass matrix final
    M = m1*(J1'*J1) + m2*(J2'*J2) + m3*(J3'*J3);

    M = M + 1e-4 * eye(3);
end


%% Gravity Vector g(q)
function grav = gravity_vector(q, g)

    q1 = q(1); q2 = q(2); q3 = q(3);
    L1=1; L2=1; L3=1;
    % Calculate the gravity vector, mi=Li=1
    g1 = g * (3* L1 * cos(q1) + 2* L2 * cos(q1 + q2) + L3 * cos(q1 + q2 + q3));
    g2 = g * (2* L2 * cos(q1 + q2) + L3 * cos(q1 + q2 + q3));
    g3 = g * L3 * cos(q1 + q2 + q3);

    %g1 = g * (3*c1 + 2*c12 + c123);
    %g2 = g * (2*c12 + c123);
    %g3 = g * c123;

    grav = [g1; g2; g3];
end

%% Coriolis/Centripetal Vector
function cor = coriolis_vector(q, q_d)

    eps = 1e-8;  %For numerical diffrentiation: perturbation
    n = 3;

    %This shows how mass matrix changes with joint angle.
    %i.e partial differentiation of M with joint angles
    dMdq = zeros(n, n, n);

    for i = 1:n
        q_plus = q;
        q_minus = q;
        q_plus(i)  = q(i) + eps;
        q_minus(i) = q(i) - eps;

        M_plus  = mass_matrix(q_plus);
        M_minus = mass_matrix(q_minus);

        dMdq(:,:,i) = (M_plus - M_minus) / (2*eps);
    end

    %Christoffel-based Coriolis vector
    cor = zeros(n, 1);

    for k = 1:n
        for i = 1:n
            for j = 1:n
                Gamma_ijk = 1/2 * ( dMdq(k,j,i) + dMdq(k,i,j) - dMdq(i,j,k));
                cor(k) = cor(k) + Gamma_ijk * q_d(i)* q_d(j);
            end
        end
    end
end
```

```
%% Forward Kinematics
function [x, y] = forward_kinematics(q)
    L1 = 1; L2 = 1; L3 = 1;

    % Base/joint 1
    x0 = 0;
    y0 = 0;


    % Joint 2
    x1 = L1*cos(q(1));
    y1 = L1*sin(q(1));

    % Joint 3
    x2 = x1 + L2*cos(q(1) + q(2));
    y2 = y1 + L2*sin(q(1) + q(2));

    % End-effector
    x3 = x2 + L3*cos(q(1) + q(2) + q(3));
    y3 = y2 + L3*sin(q(1) + q(2) + q(3));

    x = [x0, x1, x2, x3];
    y = [y0, y1, y2, y3];
end

%% Plot Results
function plot_results(t, q_hist, q_d_hist, q_dd_hist)
    figure('Name', '3R Robot Simulation Results', 'Position', [100 100 1200 800]);

    % Joint angles

    plot(t, q_hist(1,:), 'b-', 'LineWidth', 1.5); hold on;
    plot(t, q_hist(2,:), 'r-', 'LineWidth', 1.5);
    plot(t, q_hist(3,:), 'g-', 'LineWidth', 1.5);
    xlabel('Time (s)'); ylabel('Angle (rad)');
    title('Joint Angles vs Time');
    legend('\theta_1', '\theta_2', '\theta_3', 'Location', 'best');
    grid on;

    figure('Name', '3R Robot Simulation Results', 'Position', [100 100 1200 800]);
    % Joint velocities

    plot(t, q_d_hist(1,:), 'b-', 'LineWidth', 1.5); hold on;
    plot(t, q_d_hist(2,:), 'r-', 'LineWidth', 1.5);
    plot(t, q_d_hist(3,:), 'g-', 'LineWidth', 1.5);
    xlabel('Time (s)'); ylabel('Angular Velocity (rad/s)');
    title('Joint Velocities vs Time');
    legend('$\dot{\theta}_1$', '$\dot{\theta}_2$', '$\dot{\theta}_3$', ...
           'Interpreter', 'latex', 'Location', 'best');
    grid on;

    figure('Name', '3R Robot Simulation Results', 'Position', [100 100 1200 800]);
    % Joint accelerations

    plot(t, q_dd_hist(1,:), 'b-', 'LineWidth', 1.5); hold on;
    plot(t, q_dd_hist(2,:), 'r-', 'LineWidth', 1.5);
    plot(t, q_dd_hist(3,:), 'g-', 'LineWidth', 1.5);
    xlabel('Time (s)'); ylabel('Angular Acceleration (rad/s^2)');
    title('Joint Accelerations vs Time');
    legend('$\ddot{\theta}_1$', '$\ddot{\theta}_2$', '$\ddot{\theta}_3$', ...
           'Interpreter', 'latex', 'Location', 'best');
    grid on;
end
```

```
%% Animation
function animate_arm(t, q_hist, dt)
    figure('Name', '3R Robot Animation', 'Position', [100 100 800 600]);

    % Animation settings for faster simulation
    skip = 50;

    for i = 1:skip:length(t)
        [x, y] = forward_kinematics(q_hist(:,i));

        clf;
        hold on;

        % Drawing links
        plot(x, y, 'b-', 'LineWidth', 3);

        % Drawing joints
        plot(x(1), y(1), 'ko', 'MarkerSize', 12, 'MarkerFaceColor', 'k');  % Base
        plot(x(2), y(2), 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');  % Joint 2
        plot(x(3), y(3), 'ro', 'MarkerSize', 10, 'MarkerFaceColor', 'r');  % Joint 3
        plot(x(4), y(4), 'go', 'MarkerSize', 10, 'MarkerFaceColor', 'g');  % End-effector

        % Drawing masses located at link tip
        plot(x(2), y(2), 'bs', 'MarkerSize', 15, 'MarkerFaceColor', 'b');
        plot(x(3), y(3), 'bs', 'MarkerSize', 15, 'MarkerFaceColor', 'b');
        plot(x(4), y(4), 'bs', 'MarkerSize', 15, 'MarkerFaceColor', 'b');

        axis equal;
        xlim([-4 4]);
        ylim([-4 4]);
        grid on;
        xlabel('x (m)');
        ylabel('y (m)');
        title(sprintf('3R Robot Falling Under Gravity (t = %.2f s)', t(i)));
        quiver(3, 3, 0, -0.5, 'w', 'LineWidth', 2, 'MaxHeadSize', 0.5);
        text(3.2, 2.7, 'g', 'FontSize', 14);

        drawnow;
        pause(dt * skip * 0.2);
    end
end
```